

ALEXANDRE TEIXEIRA

AI Engineer · GenAI & Production LLM Systems

alexandretei13@gmail.com ◇ +33 7 86 01 55 04 ◇ **LinkedIn** ◇ **GitHub** ◇ **Portfolio** ◇ Paris, France

0→1 AI Engineer: I turn complex business problems into AI products that get deployed and adopted, from scoping the need all the way to GPU serving in production. My approach: understand the field, simplify the problem, and build from idea to impact.

Education

Université de Technologie de Compiègne (UTC) ***2022 – 2026***
Master of Engineering in Computer Science (Diplôme d'Ingénieur) Compiègne, France

- AI, optimization, operations research, logic & search-based problem solving.
- Distributed systems, parallel computing, deep learning and reinforcement learning (projects detailed below).

Shanghai University, UTSEUS (Sino-French Department) ***Sept. 2024 – Jan. 2025***
Exchange semester in Artificial Intelligence Shanghai, China

- Natural Language Processing, generative AI, data analysis, data visualization.

Professional Experience

AI Engineer, SNCF · Internship ***Sept. 2025 – Feb. 2026***
Multimodal AI platform for meeting summarization Paris, France

- **Problem:** RFIs, video calls and in-person meetings generated hours of audio that no team had time to go through.
- **Solution:** end-to-end design and build of **the platform, 0→prod**: transcription, diarization, summarization, key-slide detection and a VLM to inject visual context into meeting reports.
- Diarization on single-channel audio and **fine-tuning of a speech-to-text model** on real SNCF voices, with no golden dataset, for confidential internal meetings.
- Built the content generation, organization and delivery components; React web app with a Python FastAPI backend.
- Industrialized deployment through the group's CI/CD tooling (Docker, Jenkins) on Azure AI Services.
- Tech watch and prototyping on real-time: Realtime API (WebRTC), text-to-speech and speech-to-text models.
- **Result: 90% of audio files processed in $<$ 5 min**; platform adopted by several business teams.
- **Technical lead** for the team's AI direction: **ran an RFI across 15 AI vendors (Mistral AI, IBM, LinkUp, Vespa)** and supported the internal rollout of Mistral Code to engineering teams.

Tech stack: Python, FastAPI, React, Azure AI Services, Azure OpenAI, Docker, Jenkins, VLM, STT / TTS, WebRTC.

AI Engineer, Junior Enterprise Mission · Renault / Alpine ***Mar. 2025 – June 2025***
AI whiteboard to industrialize FMEAs and LLM infrastructure on GPU Compiègne, France

- **Problem:** FMEA risk analysis was manual and expert-dependent, which limited the number of processes covered. Target: **cut FMEA creation time from 6 months to 2 months**.
- **Solution:** designed a collaborative AI-augmented whiteboard: the LLM + RAG suggest risks, causes, effects and actions, and experts validate them.
- **Multi-agent architecture**: one agent per sub-failure of the system, with links between sub-solutions, and an orchestrator agent ensuring distinct, non-redundant suggestions.
- LLM components, vector databases and document processing pipelines to structure domain knowledge and make risk analysis more reliable.
- Chose the latest frontier model rather than running an upfront benchmark, which would have been outdated before the project ended: a stronger model was released two months later.
- **Infrastructure:** deployed open-source models on an **8× NVIDIA A100** cluster; **vLLM** inference server (serving, batching, GPU memory management) and **load balancing** across models and users.
- Exposed models through an **OpenAI-compatible API** and monitored the infrastructure: GPU availability, load, memory and resource saturation.
- Worked directly with industrial stakeholders across the Renault-Nissan and Alpine ecosystem.

Tech stack: Python, LangChain, vLLM, Ollama, PostgreSQL, vector database, Docker, NVIDIA A100.

AI Engineer, SNCF · Internship ***Sept. 2023 – Feb. 2024***
GenAI assistant automating L1 DevOps support, which became SNCF GPT Lyon, France

- **Problem:** the group was looking for where GenAI could have an impact; I identified the need in L1 DevOps support, where repetitive ServiceNow tickets were handled manually.
- **Solution:** scoped the use case and designed the RAG assistant **single-handedly from 0 to 1**, from architecture to deployment; **MVP in 3 months**.
- **RAG pipeline:** PDF / Markdown parsing, cleaning, chunking, metadata extraction and embeddings; storage on Azure Blob Storage, vector and semantic indexing with Azure Cognitive Search, generation via Azure OpenAI.
- **Agentic approach:** ReAct agent with tool-calling to retrieve information from the right source (internal documentation, past incidents, Jira, SharePoint).
- **Integrations:** ServiceNow API to contextualize answers, Microsoft Bot Framework / Azure Bot Service, Microsoft Graph API and Teams webhooks; React web interface for support teams.
- **Observability:** Langfuse for conversation tracing, auditing, LLM metrics and cost tracking; experiments with LangChain and LangGraph.
- **Industrialization:** CI/CD with GitLab, Jenkins, Ansible, Azure CLI and Vault; images and artifacts on JFrog Artifactory, Harbor and Docker Trusted Registry; Kubernetes deployment; quality and security checks with SonarQube and Checkmarx.
- **Result: 82% of L1 tickets resolved automatically**; product carried by a 25-person team and featured on BFM Business.
- The product then scaled group-wide to **become SNCF GPT** (**3,000+ monthly users**), still in service today on the architecture and agentic design I built.

  Tech stack: Python, Flask, Streamlit, React, Azure OpenAI, Azure Cognitive Search, Azure Blob Storage, Azure Web App, ServiceNow API, Microsoft Graph, Bot Framework, Langfuse, LangChain, LangGraph, Docker, Kubernetes, Terraform, Ansible, GitLab, Jenkins, Artifactory, Harbor, Vault, SonarQube, Checkmarx, Datadog, Jira.

Selected Projects

Mikky, a desktop companion to supervise coding agents ***Sept. 2026 – present***
Open-source personal project: track and supervise Claude Code and Codex without digging through terminals. **GitHub**

- **Problem:** with several coding agents running in parallel, knowing where each task stands and answering their requests means juggling between terminals.
- **Solution:** an island at the top of the screen that launches and tracks agents, shows each task's status and steps, and surfaces their permission requests with context so they can be answered in one click (Yes / No / Always).
- **Rust engine** (`mikkyd`): launches agents through their ACP adapters on Windows or WSL, follows sessions opened elsewhere by reading their session files, optional hooks, metadata in SQLite.
- **Flutter app**: transparent always-on-top window with a shader-drawn shape; communicates with the engine over a local WebSocket, secured by a token stored in the Windows credential vault.
- 100% local, no account, no telemetry; **0.013% CPU** with the island hidden; tested on replayed real Claude / Codex sessions, plus visual tests of every screen.
- **Next:** evolve Mikky into a "Discord for agents" where agents collaborate on projects, with next-generation Jira-style project management.

  Tech stack: Rust, Flutter / Dart, WebSocket, SQLite, ACP, Claude Code, Codex, WSL.

Comparative study of CNNs vs Transformers for 3D medical imaging, with UCL ***Feb. 2026 – present***
University College London: benchmarking performance, complexity and inference constraints.

- Trained and evaluated first prototype models; compared CNN and Transformer architectures on 3D medical images.
- Implemented a CNN **from scratch**: convolution, pooling, activation functions, forward pass, backpropagation and training loop.

  Tech stack: Python, PyTorch, CNN, Transformers.

Technical drawing analysis and quote estimation, metalworking plant ***Mar. – June 2025***
SaaS platform: computer vision + machine learning to estimate part manufacturing time.

- **Problem:** part pricing relied on manually reading technical drawings.
- Automatic data extraction from drawings with computer vision (YOLOv8, Transformers), then an ML model predicting manufacturing time and the quote.
- Full value chain: extraction, processing, storage and delivery through a web interface.
- Supported non-technical users in building a quote history large enough to train the model.

  Tech stack: Python, PyTorch, YOLOv8, Transformers, PostgreSQL, Next.js.

On-premise LLM platform for students (UTC) ***Feb. – July 2025***
OpenWebUI + Ollama on an RTX 5090, hosted on UTC servers and open to all students.

- Deployed and operated the platform; main constraint: fitting the models into the server's VRAM.

  Tech stack: OpenWebUI, Ollama, NVIDIA RTX 5090.

AlphaZero from scratch, Gopher & Dodo games *2024*
MCTS, self-play and deep RL; residual network in PyTorch with policy and value heads.

- Monte Carlo Tree Search combined with a ResNet; training loop through automatic game generation, policy loss, value loss and model checkpointing.
- Parallelization to speed up training and MCTS search.

Tech stack: Python, PyTorch.

Decentralized distributed system with data verification *2025*
P2P network in Go: sensor, verifier and user nodes.

- Local replicas, logical clocks, distributed locks, distributed queue, peer discovery and P2P message propagation.
- Simulated sensor data verification with consistency across replicas and user-side prediction from local data.

Tech stack: Go.

Geospatial analysis of Shanghai's river network *2024 – 2025*
Modeling rivers and canals as a graph to study connectivity and flows.

- Extracted and cleaned waterways from OpenStreetMap; enriched geometries, segments, intersections and potential flow directions.
- Built the waterway graph, geospatial features for ML models and first flow prediction approaches; Parquet storage for faster reads / writes than CSV.

Tech stack: Python, Pandas, GeoPandas, OpenStreetMap, Parquet.

Search engine and visualization, Game of Thrones characters *2025*
Scraping, text processing, NLP and relationship graph generation.

- Search engine over a scraped corpus and interactive visualization of character relationships.

Tech stack: Python, NLP, scraping, graphs.

Research Experience

Laboratoire des Technologies Innovantes (LTI), UPJV Amiens *2022*
Deep learning for building energy management.

- LSTM models on sensor data (CO_2, noise) to detect occupancy and control energy consumption.

UTC Laboratory, parallel computing *2025*
Small-scale high-performance computing.

- Solved Poisson equations on a mini supercomputer to evaluate the hardware before moving to larger systems (GENCI / Jean Zay).

Technical Skills

AI / LLM Engineering: PyTorch, TensorFlow, Hugging Face, LangChain, LangGraph, Pydantic, RAG, embeddings, vector databases, VLM, STT / TTS, fine-tuning
Agents & LLMOps: ReAct agents, tool-calling, multi-agent architectures, coding agent orchestration (Claude Code, Codex, ACP), Langfuse (tracing, auditing, costs), LLM evaluation
Serving & GPU infra: vLLM, Ollama, OpenWebUI, batching, GPU memory management, load balancing, OpenAI-compatible API, NVIDIA A100 / RTX 5090
Machine Learning: CNNs, Transformers, ResNet, LSTM, YOLOv8, MCTS / RL, Scikit-learn, model training & evaluation
Cloud / DevOps: Azure AI Services, Azure OpenAI, Cognitive Search, Blob Storage, Docker, Kubernetes, Terraform, Ansible, Vault
CI/CD & quality: GitLab CI/CD, Jenkins, JFrog Artifactory, Harbor, SonarQube, Checkmarx, Datadog
Integrations: ServiceNow API, Microsoft Graph API, Bot Framework, Teams webhooks, REST APIs
Development: Python, Rust, Go, C / C++, Dart / Flutter, SQL, React, Next.js, FastAPI, Flask, Streamlit, PostgreSQL, SQLite, WebSocket
Data & scientific computing: Pandas, NumPy, GeoPandas, Parquet, Matplotlib, R, Matlab, Scilab

Languages & Leadership

Languages : French (native) · English (professional) · Chinese (basic).
Head of Partnerships, Imaginarium Festival : student festival, EUR 600,000 in sponsorship. *2022 – 2023*
Founder Breakfast, Shanghai : discussions with startup founders on AI innovation. *2024 – 2025*
Fruit & vegetable markets : working with the family business since age 14.
Interests : swimming, running, video games, new technologies.